FPT Education

FPT POLYTECHNIC



cdio™

Conceive Design Implement Operate

BÀI 5. THỰC HIỆN DỰ ÁN 3

Giảng viên:

https://caodang.fpt.edu.vn

MỤC TIÊU

- Xây dựng chức năng quản lý sản phẩm (CRUD)
- Xây dựng chức năng quản lý nhân viên (CRUD)



QUẢN LÝ SẢN PHẨM

- Chức năng quản lý sản phẩm cho phép hệ thống quản lý thức uống, đảm bảo thông tin chính xác, nhất quán và sẵn sàng phục vụ hệ thống bán hàng
- Nghiệp vụ chính:
  - Thêm, cập nhật, xoá sản phẩm
  - Quản lý giá bán, hình ảnh, trạng thái kinh doanh
  - Phân quyền sử dụng
    - Admin: toàn quyền quản lý
    - Users: Chỉ xem danh sách sản phẩm đang bán

QUẢN LÝ NHÂN VIÊN

- Chức năng quản lý nhân viên giúp kiểm soát thông tin và quyền truy cập của đội ngũ vận hành, đảm bảo phân quyền rõ ràng và an toàn hệ thống
- Nghiệp vụ chính:
  - Thêm, cập nhật, khoá tài khoản nhân viên
  - Chỉ Admin được phép quản lý nhân sự



XÂY DỰNG CHỨC NĂNG QUẢN LÝ ĐỒ UỐNG (CRUD)

YÊU CẦU NGHIỆP VỤ

- Quản lý đồ uống (CRUD) là chức năng để người dùng trong hệ thống có thể quản lý các thông tin của đồ uống đang kinh doanh
- Yêu cầu nghiệp vụ chức năng và đối tượng sử dụng:
  - Xem đồ uống: Quản lý
  - Thêm đồ uống: Quản lý
  - Cập nhật đồ uống: Quản lý
  - Xoá đồ uống: Quản lý

THÔNG TIN ĐỒ UỐNG

Cơ sở dữ liệu

dbo.drinks		
int	id	PK
int	category_id	FK
nvarchar(250)	name	
int	price	
nvarchar(250)	image	
nvarchar(-1)	description	
bit	active	

Entity Class

```
public class DrinkDAL
{
    4 references
    public int Id { get; set; }
    3 references
    public int CategoryId { get; set; }
    3 references
    public string? Name { get; set; }
    3 references
    public int Price { get; set; }
    3 references
    public string? Image { get; set; }
    3 references
    public string? Description { get; set; }
    3 references
    public bool Active { get; set; }
```

Các phương thức cần thiết trong ***DrinkDAL***

- Phương thức chuyển dữ liệu sang Entity
- Phương thức lấy danh sách tất cả
- Phương thức lấy danh sách theo loại
- Phương thức lấy thông tin theo id
- Phương thức thêm mới
- Phương thức cập nhật
- Phương thức xoá

```
3 references
private static DrinkDAL FromDataRow(DataRow row)...

0 references
public static List<DrinkDAL> GetAll()...

0 references
public static List<DrinkDAL> GetByCategory(int categoryId)...

0 references
public static DrinkDAL? GetById(int id)...

0 references
public static int Insert(DrinkDAL drink)...

0 references
public static bool Update(DrinkDAL drink)...

0 references
public static bool Delete(int id)...
```

DrinkController

Cài đặt mã cho ***DrinkController***

```
[Authorize]
0 references
public class DrinksController : Controller
{
    [Authorize(Policy = "EmployeeOrManager")]
    3 references
    public IActionResult Index()...

    [Authorize(Policy = "EmployeeOrManager")]
    0 references
    public IActionResult Details(int id)...

    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Create()...

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Create(DrinkDAL drink)...

    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Edit(int id)...

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Edit(int id, DrinkDAL drink)...

    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Delete(int id)...

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult DeleteConfirmed(int id)...
}
```

Cài đặt phân quyền

Bảo mật CSRF

DrinkController

- Cài đặt mã cho ***Index*** (lấy danh sách tất cả đồ uống) và ***Details*** (xem chi tiết)

```
[Authorize(Policy = "EmployeeOrManager")]
public IActionResult Index()
{
    var list = DrinkDAL.GetAll();
    return View(list);
}

[Authorize(Policy = "EmployeeOrManager")]
public IActionResult Details(int id)
{
    var drink = DrinkDAL.GetById(id);
    if (drink == null) return NotFound();
    var category = CategoryDAL.GetById(drink.CategoryId);
    ViewBag.CategoryName = category?.Name ?? string.Empty;
    return View(drink);
}
```

❑ Giao diện quản lý danh sách đồ uống

Danh sách đồ uống

Thêm mới

ID	Tên	Mô tả	Ảnh	Giá (VNĐ)	Hiển thị	Hành động
1	Cà phê sữa	Đậm vị, thơm mùi sữa		45.000	Có	Sửa Xóa
2	Trà đào	Hương đào tươi, mát dịu		50.000	Có	Sửa Xóa
3	Bạc xỉu	Ngọt dịu, nhẹ nhàng		40.000	Không	Sửa Xóa
4	Sinh tố bơ	Béo mịn, giàu dinh dưỡng		55.000	Có	Sửa Xóa
5	Matcha Latte	Vị matcha nguyên chất, thơm trọn vị		60.000	Có	Sửa Xóa

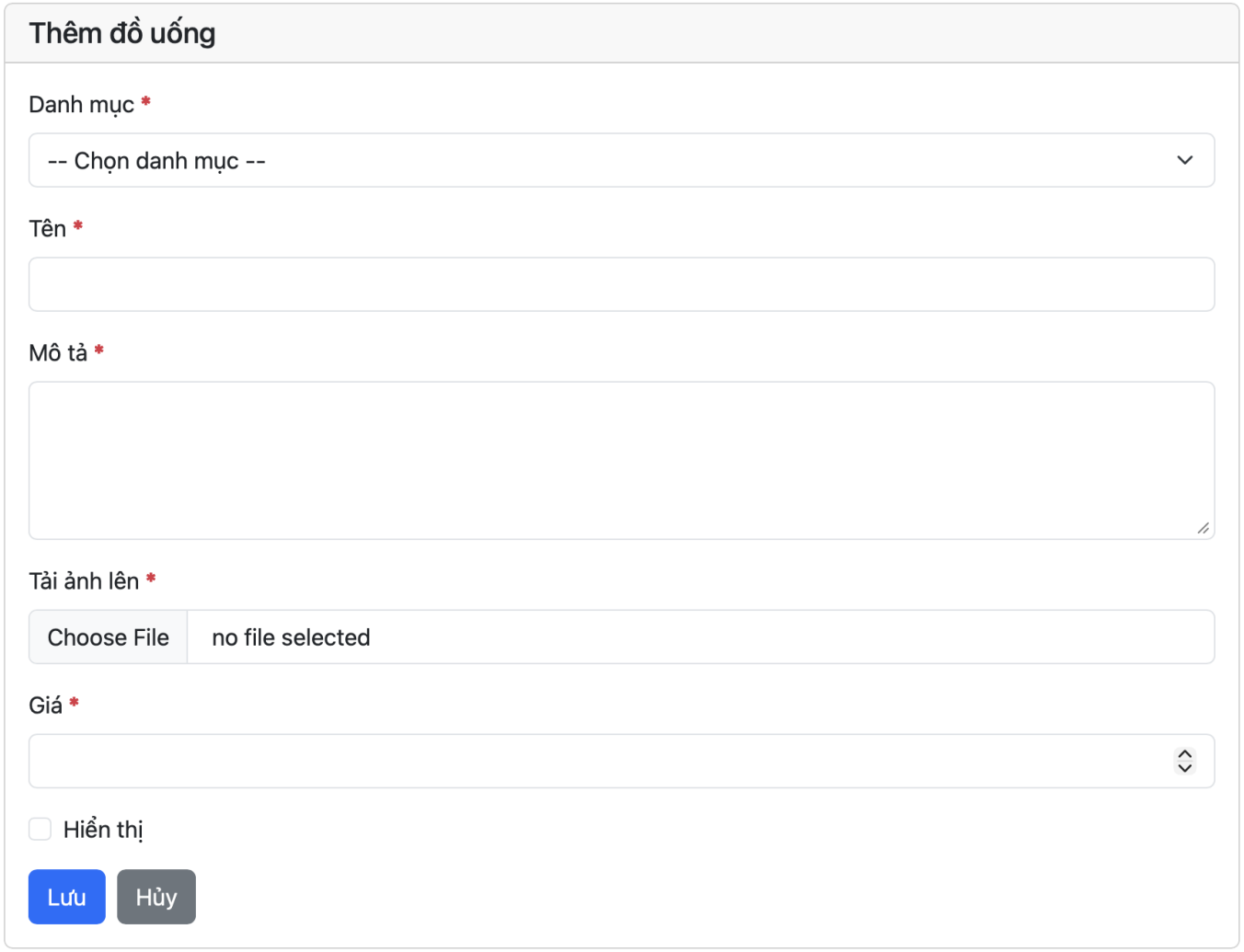
Xử lý upload hoặc xóa ảnh bằng Ajax (View Create và Update)

```
fileInput.addEventListener('change', async function (e) {
    const f = fileInput.files && fileInput.files[0];
    if (!f) return;
    // Upload via AJAX to /File/Upload
    const fd = new FormData();
    fd.append('file', f);
    fd.append('__RequestVerificationToken', getAntiForgeryToken());
    try {
        const resp = await fetch('@Url.Action("Upload", "File")', {
            method: 'POST',
            body: fd
        });
        const json = await resp.json();
        if (json && json.success) {
            hiddenInput.value = json.fileName || '';
            preview.src = json.url || ('/uploads/' + json.fileName);
            preview.style.display = 'inline-block';
            btnRemove.style.display = 'inline-block';
        } else {
            alert(json?.error || 'Upload failed');
        }
    } catch (err) {
        console.error(err);
        alert('Upload failed');
    } finally {
        fileInput.value = '';
    }
});
```

Xử lý upload hoặc xóa ảnh bằng Ajax (View Create và Update)

```
btnRemove.addEventListener('click', async function () {
    const fileName = hiddenInput.value;
    if (!fileName) {
        preview.src = '';
        preview.style.display = 'none';
        btnRemove.style.display = 'none';
        return;
    }
    const fd = new FormData();
    fd.append('fileName', fileName);
    fd.append('__RequestVerificationToken', getAntiForgeryToken());
    try {
        const resp = await fetch('@Url.Action("Delete", "File")', {
            method: 'POST',
            body: fd
        });
        const json = await resp.json();
        if (json && json.success) {
            hiddenInput.value = '';
            preview.src = '';
            preview.style.display = 'none';
            btnRemove.style.display = 'none';
        } else {
            alert(json?.error || 'Delete failed');
        }
    } catch (err) {
        console.error(err);
        alert('Delete failed');
    }
});
```

Giao diện form đồ uống



DrinkController

- Xây dựng phương thức **Create** (HttpGET) và **Create** (HttpPOST)

```
[Authorize(Policy = "ManagerOnly")]
public IActionResult Create()
{
    ViewBag.Categories = CategoryDAL.GetAll();
    return View(new DrinkDAL());
}
```

Gọi giao diện của trang thêm mới

Xử lý form sau khi submit

```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult Create(DrinkDAL drink)
{
    //xử lý validate dữ liệu đầu vào

    DrinkDAL.Insert(drink);
    return RedirectToAction(nameof(Index));
}
```

DrinkController

- Xây dựng phương thức **Create** (HttpGET) và **Create** (HttpPOST)

```
//xử lý validate dữ liệu đầu vào
if (drink == null) return BadRequest();

if (string.IsNullOrWhiteSpace(drink.Name))
    ModelState.AddModelError(nameof(drink.Name), "Name is required.");

if (drink.CategoryId <= 0)
    ModelState.AddModelError(nameof(drink.CategoryId), "Category is required.");

if (drink.Price < 0)
    ModelState.AddModelError(nameof(drink.Price), "Price must be 0 or greater.");

if (!ModelState.IsValid)
{
    ViewBag.Categories = CategoryDAL.GetAll();
    return View(drink);
}
```

DrinkController

- Xây dựng phương thức **Update** (HttpGET) và **Update** (HttpPOST)

```
[Authorize(Policy = "ManagerOnly")]
public IActionResult Edit(int id)
{
    var drink = DrinkDAL.GetById(id);
    if (drink == null) return NotFound();
    ViewBag.Categories = CategoryDAL.GetAll();
    return View(drink);
}
```

Gọi giao diện của trang chỉnh sửa

Xử lý form sau khi submit

```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult Edit(int id, DrinkDAL drink)
{
    //xử lý validate dữ liệu đầu vào
    var ok = DrinkDAL.Update(drink);
    if (!ok) ModelState.AddModelError(string.Empty, "Update failed.");
    if (!ModelState.IsValid)
    {
        ViewBag.Categories = CategoryDAL.GetAll();
        return View(drink);
    }
    return RedirectToAction(nameof(Index));
}
```

DrinkController

- Xây dựng phương thức **Update** (HttpGET) và **Update** (HttpPOST)

```
//xử lý validate dữ liệu đầu vào
if (drink == null) return BadRequest();
if (id != drink.Id) return BadRequest();
if (string.IsNullOrWhiteSpace(drink.Name))
    ModelState.AddModelError(nameof(drink.Name), "Name is required.");
if (drink.CategoryId <= 0)
    ModelState.AddModelError(nameof(drink.CategoryId), "Category is required.");
if (drink.Price < 0)
    ModelState.AddModelError(nameof(drink.Price), "Price must be 0 or greater.");
if (!ModelState.IsValid)
{
    ViewBag.Categories = CategoryDAL.GetAll();
    return View(drink);
}
```

DrinkController

- Xây dựng phương thức **Delete**.

```
[Authorize(Policy = "ManagerOnly")]
public IActionResult Delete(int id)
{
    var drink = DrinkDAL.GetById(id);
    if (drink == null) return NotFound();
    var category = CategoryDAL.GetById(drink.CategoryId);
    ViewBag.CategoryName = category?.Name ?? string.Empty;
    return View(drink);
}
```

Gọi giao diện xác nhận xóa

```
[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult DeleteConfirmed(int id)
{
    var ok = DrinkDAL.Delete(id);
    if (!ok) return BadRequest();
    return RedirectToAction(nameof(Index));
}
```

Xử lý xóa khi đã xác nhận



XÂY DỰNG CHỨC NĂNG QUẢN LÝ NHÂN VIÊN (CRUD)

YÊU CẦU NGHIỆP VỤ

- Quản lý nhân viên (CRUD) là chức năng để người dùng người quản trị có thể quản lý các tài khoản nhân viên sử dụng hệ thống
- Yêu cầu nghiệp vụ chức năng và đối tượng sử dụng:
  - Xem danh sách nhân viên: Admin
  - Thêm nhân viên: Admin
  - Cập nhật nhân viên: Admin
  - Khoá tài khoản nhân viên: Admin

THÔNG TIN TÀI KHOẢN

Cơ sở dữ liệu

dbo.users		
int	id	PK
varchar(100)	email	
varchar(255)	password	
nvarchar(60)	full_name	
varchar(10)	phone	
bit	role	
bit	active	



Entity Class

```
public class UserDAL
{
    5 references
    public int Id { get; set; }
    5 references
    public string? Email { get; set; }
    3 references
    public string? Password { get; set; }
    4 references
    public string? FullName { get; set; }
    3 references
    public string? Phone { get; set; }
    4 references
    public bool Role { get; set; }
    3 references
    public bool Active { get; set; }
```

Các phương thức cần thiết trong ***UserDAL***

Phương thức lấy thông tin theo id

Phương thức tìm tài khoản theo email

Phương thức kiểm tra đăng nhập

Phương thức thêm mới

Phương thức cập nhật

Phương thức xóa

```
4 references
private static UserDAL FromDataRow(DataRow row)...

0 references
public static List<UserDAL> GetAll()...

0 references
public static UserDAL? GetById(int id)...

0 references
public static UserDAL? GetByEmail(string email)...

1 reference
public static UserDAL? Authenticate(string email, string password)...

0 references
public static int Insert(UserDAL user)...

0 references
public static bool Update(UserDAL user)...

0 references
public static bool Delete(int id)...
```

UsersController

Cài đặt mã cho *UsersController*

```
[Authorize]
0 references
public class UsersController : Controller
{
    [Authorize(Policy = "EmployeeOrManager")]
    3 references
    public IActionResult Index()...

    [Authorize(Policy = "EmployeeOrManager")]
    0 references
    public IActionResult Details(int id)...
```

Quyền xem

Quyền thêm, sửa, xóa

```
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Create()...

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Create(UserDAL user)...

    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Edit(int id)...

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Edit(int id, UserDAL user)...

    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult Delete(int id)...

    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize(Policy = "ManagerOnly")]
    0 references
    public IActionResult DeleteConfirmed(int id)...
}
```

Giao diện quản lý danh sách nhân viên

Danh sách nhân viên

Thêm mới

Mã	Email	Họ và tên	Số điện thoại	Trạng thái	Hành động
1	nguyenvana@example.com	Nguyễn Văn A	0909123456	Hoạt động	Cập nhật Khóa tài khoản
2	lethib@example.com	Lê Thị B	0912345678	Không hoạt động	Cập nhật Mở khóa
3	tranvan@example.com	Trần Văn C	0987654321	Hoạt động	Cập nhật Khóa tài khoản
4	phamth@example.com	Phạm Thị D	0933222111	Không hoạt động	Cập nhật Mở khóa
5	ngocanh@example.com	Ngọc Ánh	0977001122	Hoạt động	Cập nhật Khóa tài khoản

UsersController

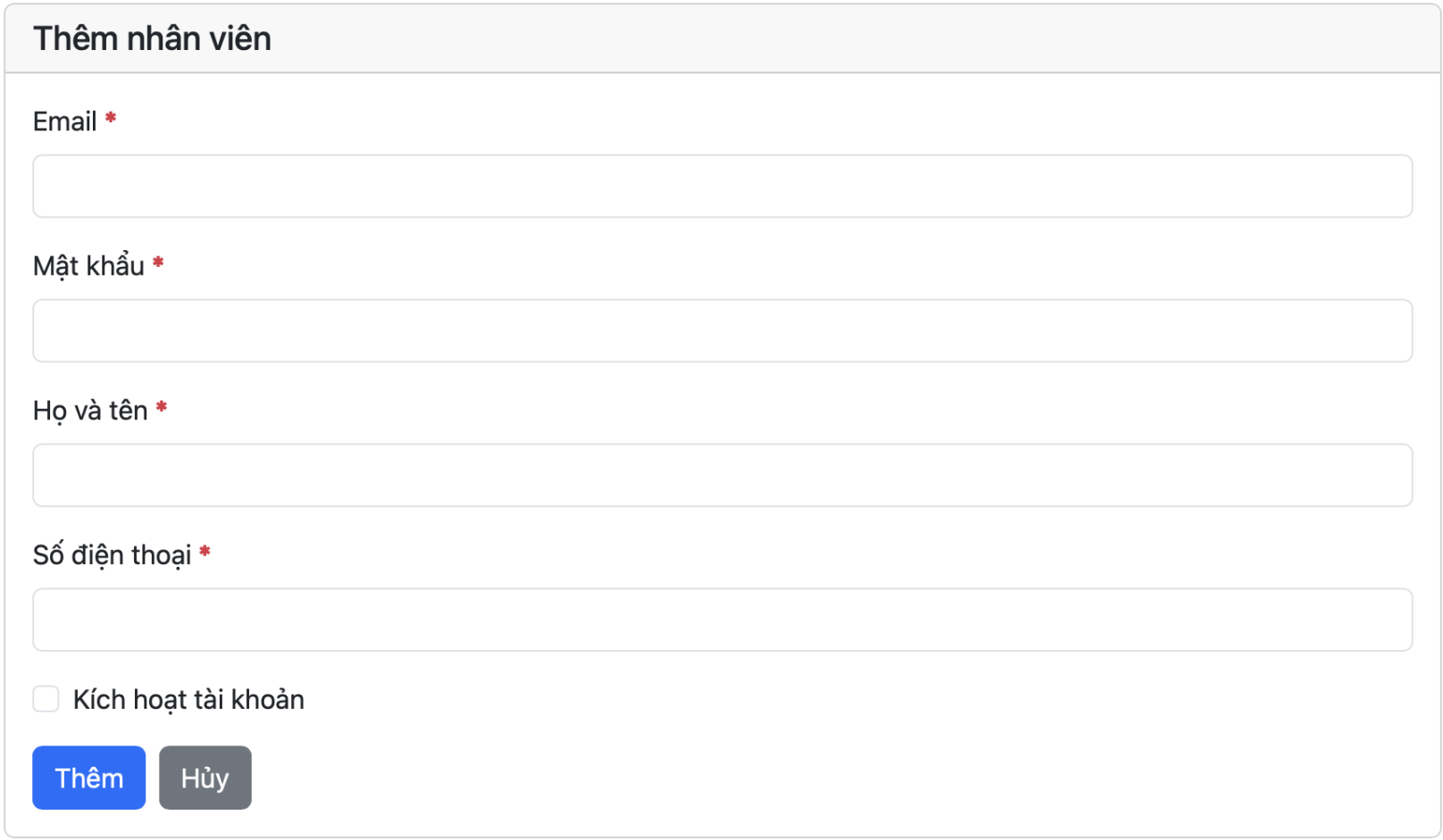
- Xây dựng phương thức **lấy danh sách tài khoản nhân viên và cập nhật trạng thái tài khoản**.
  - Chỉ lấy danh sách các tài khoản có role == false (hoặc 0) – giới hạn ở DAL
  - Chỉ được cập nhật trạng thái tài khoản không được xoá tài khoản

UsersController

```
[Authorize(Policy = "EmployeeOrManager")]
public IActionResult Index()
{
    var list = UserDAL.GetAll();
    return View(list);
}


[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult SetActive(int id, bool active)
{
    var ok = UserDAL.UpdateActive(id, active);
    var requestedWith = Request.Headers["X-Requested-With"].ToString();
    if (string.Equals(requestedWith, "XMLHttpRequest", StringComparison.OrdinalIgnoreCase))
    {
        return Ok(new { success = ok });
    }
    if (!ok) return BadRequest();
    return RedirectToAction(nameof(Index));
}
```

Giao diện thêm nhân viên



UsersController

- Xây dựng phương thức **Create** (HttpGet) và **Create** (HttpPost).

```
[Authorize(Policy = "ManagerOnly")]
public IActionResult Create()
{
    return View(new UserDAL());
}

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult Create(UserDAL user)
{
    //Validation

    if (!ModelState.IsValid)
        return View(user);
    UserDAL.Insert(user);
    return RedirectToAction(nameof(Index));
}
```

UsersController

```
//Validation
if (user == null) return BadRequest();

if (string.IsNullOrWhiteSpace(user.Email))
    ModelState.AddModelError(nameof(user.Email), "Email is required.");

if (string.IsNullOrWhiteSpace(user.Password))
    ModelState.AddModelError(nameof(user.Password), "Password is required.");

if (string.IsNullOrWhiteSpace(user.FullName))
    ModelState.AddModelError(nameof(user.FullName), "Full name is required.");

if (!ModelState.IsValid)
    return View(user);
```

UsersController

- Xây dựng phương thức **Edit** (HttpGet) và **Edit** (HttpPost).

```
[Authorize(Policy = "ManagerOnly")]
        public IActionResult Edit(int id)
        {
            var user = UserDAL.GetById(id);
            if (user == null) return NotFound();
            return View(user);
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        [Authorize(Policy = "ManagerOnly")]
        public IActionResult Edit(int id, UserDAL user)
        {
            //Validation

            var ok = UserDAL.Update(user);
            if (!ok) ModelState.AddModelError(string.Empty, "Update failed.");
            if (!ModelState.IsValid) return View(user);

            return RedirectToAction(nameof(Index));
        }
```

UsersController

```
//Validation
if (user == null) return BadRequest();
if (id != user.Id) return BadRequest();

if (string.IsNullOrWhiteSpace(user.Email))
    ModelState.AddModelError(nameof(user.Email), "Email is required.");

if (string.IsNullOrWhiteSpace(user.FullName))
    ModelState.AddModelError(nameof(user.FullName), "Full name is required.");

if (!ModelState.IsValid)
    return View(user);

var existing = UserDAL.GetById(id);
if (existing == null) return NotFound();
if (string.IsNullOrWhiteSpace(user.Password))
    user.Password = existing.Password;
```

UsersController

- Xây dựng phương thức **Detele** và **ConfirmDelete**.

```
public IActionResult Delete(int id)
{
    var user = UserDAL.GetById(id);
    if (user == null) return NotFound();
    return View(user);
}

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Policy = "ManagerOnly")]
public IActionResult DeleteConfirmed(int id)
{
    var ok = UserDAL.Delete(id);
    if (!ok) return BadRequest();
    return RedirectToAction(nameof(Index));
}
```

Summary

- Xây dựng chức năng quản lý sản phẩm (CRUD)
- Xây dựng chức năng quản lý nhân viên (CRUD)




FPT Education
FPT POLYTECHNIC
Thank you